

Notes on Ch 17: Dates and Times

Norman Lim

2025-07-30

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr    1.5.1
## v ggplot2    3.5.2      v tibble     3.3.0
## v lubridate  1.9.4      v tidyr      1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(nycflights13)
```

- Getting the current date and time using `today()` and `now()`:

```
today()
```

```
## [1] "2025-08-03"
```

```
now()
```

```
## [1] "2025-08-03 17:11:10 PST"
```

Other common ways of creating a date/time (aside from `now()` and `today()`): - While reading a file with `readr`.

- From a string.
- From individual date-time components.
- From an existing date/time object.

During import

- `readr` will automatically recognize ISO8601 date/date-time formats:

```
csv <- "
  date,datetime
  2022-01-02,2022-01-02 05:12
"
```

```
read_csv(csv)
```

```
## Rows: 1 Columns: 2
```

```
## -- Column specification -----
```

```
## Delimiter: ",",
```

```
## dtm (1): datetime
## date (1): date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

## # A tibble: 1 x 2
##   date      datetime
##   <date>    <dtm>
## 1 2022-01-02 2022-01-02 05:12:00
```

Other non-ISO8601 datetime formats can be read using `cols()` and `col_date()`:

```
csv <- "
  date
  01/02/15
"

read_csv(csv, col_types = cols(date = col_date("%d/%m/%y")))
```

```
## # A tibble: 1 x 1
##   date
##   <date>
## 1 2015-02-01

read_csv(csv, col_types = cols(date = col_date("%d/%m/%y")))
```

```
## # A tibble: 1 x 1
##   date
##   <date>
## 1 2015-02-01

read_csv(csv, col_types = cols(date = col_date("%y/%m/%d")))
```

```
## # A tibble: 1 x 1
##   date
##   <date>
## 1 2001-02-15
```

- When using `%b` or `%B` for non-English dates, a `locale()` needs to be specified.

From strings

```
# dates only
ymd("2017-01-31")

## [1] "2017-01-31"

mdy("January 31st, 2017")

## [1] "2017-01-31"

dmy("31-Jan 2017")

## [1] "2017-01-31"

# date-time
ymd_hms("2017-01-31 20:11:59")

## [1] "2017-01-31 20:11:59 UTC"
```

```
mdy_hm("01/31/2017 08:01")
```

```
## [1] "2017-01-31 08:01:00 UTC"
```

- date-time with timezone

```
ymd("2017-01-31", tz = "UTC")
```

```
## [1] "2017-01-31 UTC"
```

From individual components

- When date-time components are scattered across columns, use `make_datetime()`:

```
flights |>
  select(year, month, day, hour, minute) |>
  mutate(departure = make_datetime(year, month, day, hour, minute))
```

```
## # A tibble: 336,776 x 6
```

```
##   year month   day hour minute departure
##   <int> <int> <int> <dbl> <dbl> <dtm>
## 1  2013     1     1     5     15 2013-01-01 05:15:00
## 2  2013     1     1     5     29 2013-01-01 05:29:00
## 3  2013     1     1     5     40 2013-01-01 05:40:00
## 4  2013     1     1     5     45 2013-01-01 05:45:00
## 5  2013     1     1     6      0 2013-01-01 06:00:00
## 6  2013     1     1     5     58 2013-01-01 05:58:00
## 7  2013     1     1     6      0 2013-01-01 06:00:00
## 8  2013     1     1     6      0 2013-01-01 06:00:00
## 9  2013     1     1     6      0 2013-01-01 06:00:00
## 10 2013     1     1     6      0 2013-01-01 06:00:00
```

```
## # i 336,766 more rows
```

- making a function for creating date-time from components:

```
make_datetime_100 <- function(year, month, day, time) {
  make_datetime(year, month, day, time %% 100, time %% 100)
}
```

Using the function to create date-time for all the time columns in `flights`:

```
flights_dt <- flights |>
  filter(!is.na(dep_time), !is.na(arr_time)) |>
  mutate(
    dep_time = make_datetime_100(year, month, day, dep_time),
    arr_time = make_datetime_100(year, month, day, arr_time),
    sched_dep_time = make_datetime_100(year, month, day, sched_dep_time),
    sched_arr_time = make_datetime_100(year, month, day, sched_arr_time)
  ) |>
  select(origin, dest, ends_with("delay"), ends_with("time"))
```

```
flights_dt
```

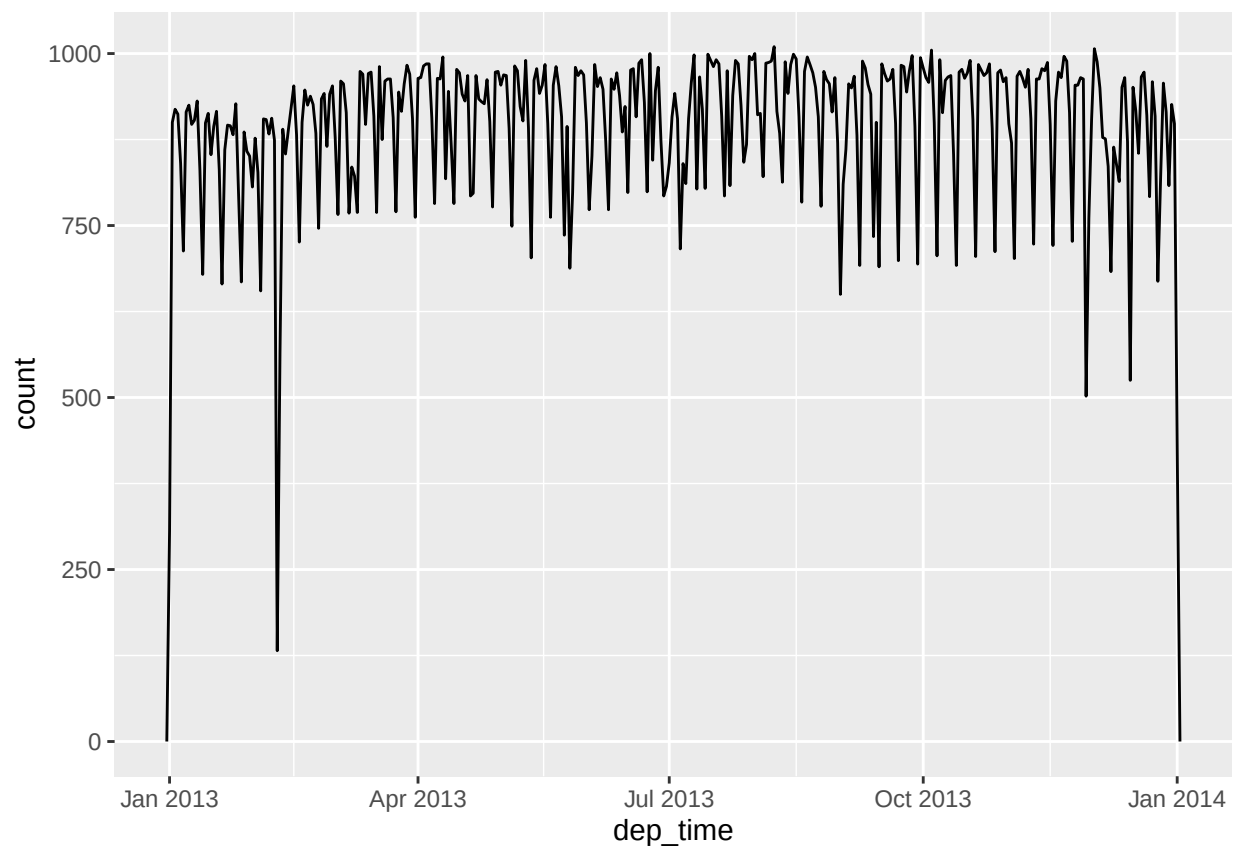
```
## # A tibble: 328,063 x 9
```

```
##   origin dest dep_delay arr_delay dep_time          sched_dep_time
##   <chr>  <chr>      <dbl>      <dbl> <dtm>          <dtm>
## 1 EWR    IAH         2        11 2013-01-01 05:17:00 2013-01-01 05:15:00
## 2 LGA    IAH         4        20 2013-01-01 05:33:00 2013-01-01 05:29:00
```

```
## 3 JFK MIA 2 33 2013-01-01 05:42:00 2013-01-01 05:40:00
## 4 JFK BQN -1 -18 2013-01-01 05:44:00 2013-01-01 05:45:00
## 5 LGA ATL -6 -25 2013-01-01 05:54:00 2013-01-01 06:00:00
## 6 EWR ORD -4 12 2013-01-01 05:54:00 2013-01-01 05:58:00
## 7 EWR FLL -5 19 2013-01-01 05:55:00 2013-01-01 06:00:00
## 8 LGA IAD -3 -14 2013-01-01 05:57:00 2013-01-01 06:00:00
## 9 JFK MCO -3 -8 2013-01-01 05:57:00 2013-01-01 06:00:00
## 10 LGA ORD -2 8 2013-01-01 05:58:00 2013-01-01 06:00:00
## # i 328,053 more rows
## # i 3 more variables: arr_time <dtm>, sched_arr_time <dtm>, air_time <dbl>
```

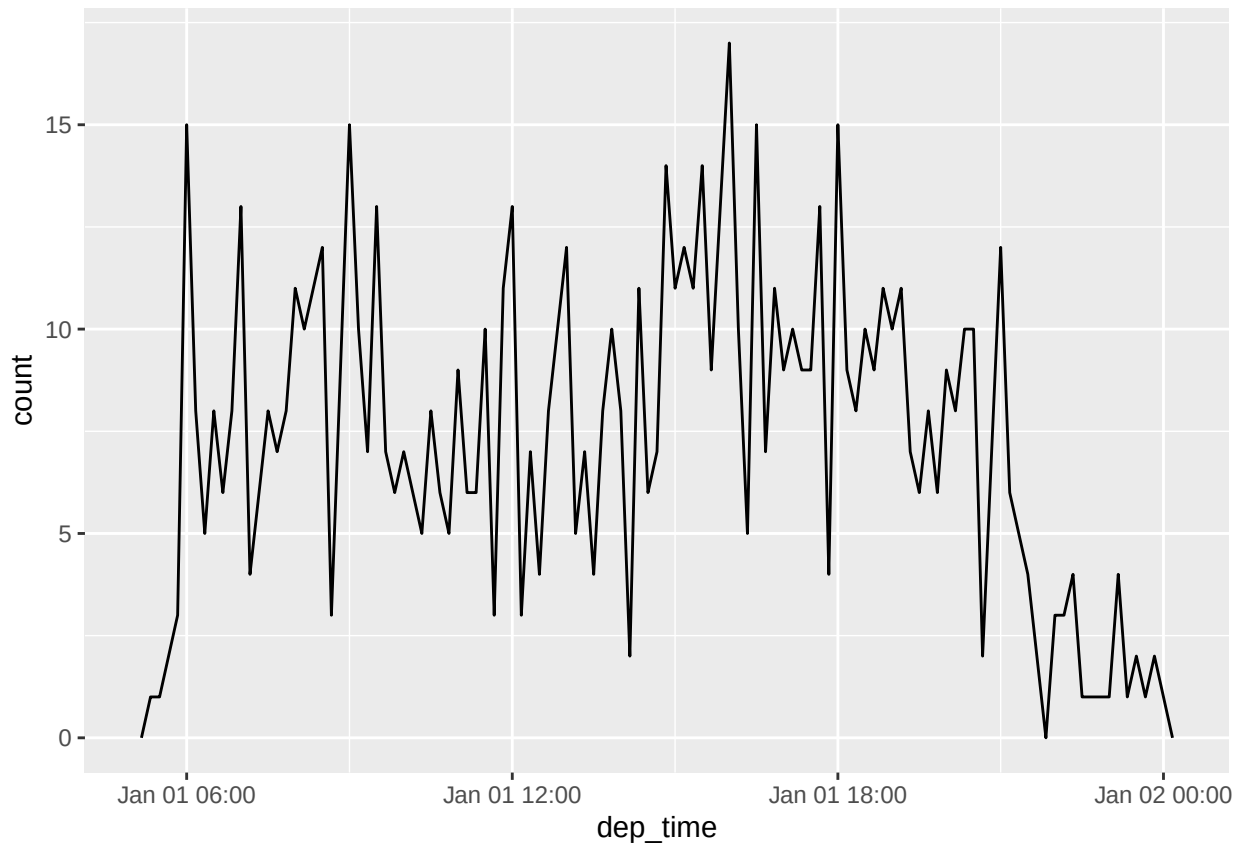
Visualizing the distribution of departure times across the year:

```
flights_dt |>
  ggplot(aes(x = dep_time)) +
  geom_freqpoly(binwidth = 86400) # seconds in a day
```



Visualizing the distribution of departure times within a single day:

```
flights_dt |>
  filter(dep_time < ymd(20130102)) |>
  ggplot(aes(x = dep_time)) +
  geom_freqpoly(binwidth = 600) # 600s = 10 minutes
```



From other types

- Use `as_datetime()` and `as_date()`:

```
as_datetime(today())
```

```
## [1] "2025-08-03 UTC"
```

```
as_date(now())
```

```
## [1] "2025-08-03"
```

- When date/times are expressed as numeric offsets from “Unix Epoch” of 1970-01-01; if the offset is in seconds, use `as_datetime()`. if the offset is in days, use `as_date()`.

```
as_datetime(60 * 60 * 10)
```

```
## [1] "1970-01-01 10:00:00 UTC"
```

```
as_date(365 * 10 + 2)
```

```
## [1] "1980-01-01"
```

Exercises

1. What happens if you parse a string that contains invalid dates?

```
ymd(c("2010-10-10", "bananas"))
```

```
## Warning: 1 failed to parse.
```

```
## [1] "2010-10-10" NA
```

2. What does the `tzzone` argument `today()` do? Why is it important?

Ans. As the comic-book character Calvin remarked, “Don’t worry about the world ending today – it is already”tomorrow” in Australia.”

3. For each of the following date-times, show ho you’d parse it using a readr column specification and a lubridate function.

```
d1 <- "January 1, 2010"
d2 <- "2015-Mar-07"
d3 <- "06-Jun-2017"
d4 <- c("August 19 (2015)", "July 1 (2015)")
d5 <- "12/30/14" # Dec 30, 2014
t1 <- "1705"
t2 <- "11:15:10.12 PM"
```

```
mdy(d1)
```

```
## [1] "2010-01-01"
```

```
ymd(d2)
```

```
## [1] "2015-03-07"
```

```
dmy(d3)
```

```
## [1] "2017-06-06"
```

```
mdy(d4)
```

```
## [1] "2015-08-19" "2015-07-01"
```

```
mdy(d5)
```

```
## [1] "2014-12-30"
```

```
parse_time(t1, "%H%M")
```

```
## 17:05:00
```

```
hms(t2)
```

```
## [1] "11H 15M 10.12S"
```

Date-time components

Getting components

Various commands are available: - year: `year()`

- month: `month()`

- day of month: `mday()`

- day of year: `yday()`

- day of week: `wday()`

- hour: `hour()`

- minute: `minute()`

- second: `second()`

Example:

```
datetime <- ymd_hms("2026-07-08 12:34:56")
```

```
year(datetime)
```

```
## [1] 2026
```

```
month(datetime)
```

```
## [1] 7
```

```
month(datetime, label = TRUE)
```

```
## [1] Jul
```

```
## 12 Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < ... < Dec
```

```
month(datetime, label = TRUE, abbr = TRUE)
```

```
## [1] Jul
```

```
## 12 Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < ... < Dec
```

```
mday(datetime)
```

```
## [1] 8
```

```
yday(datetime)
```

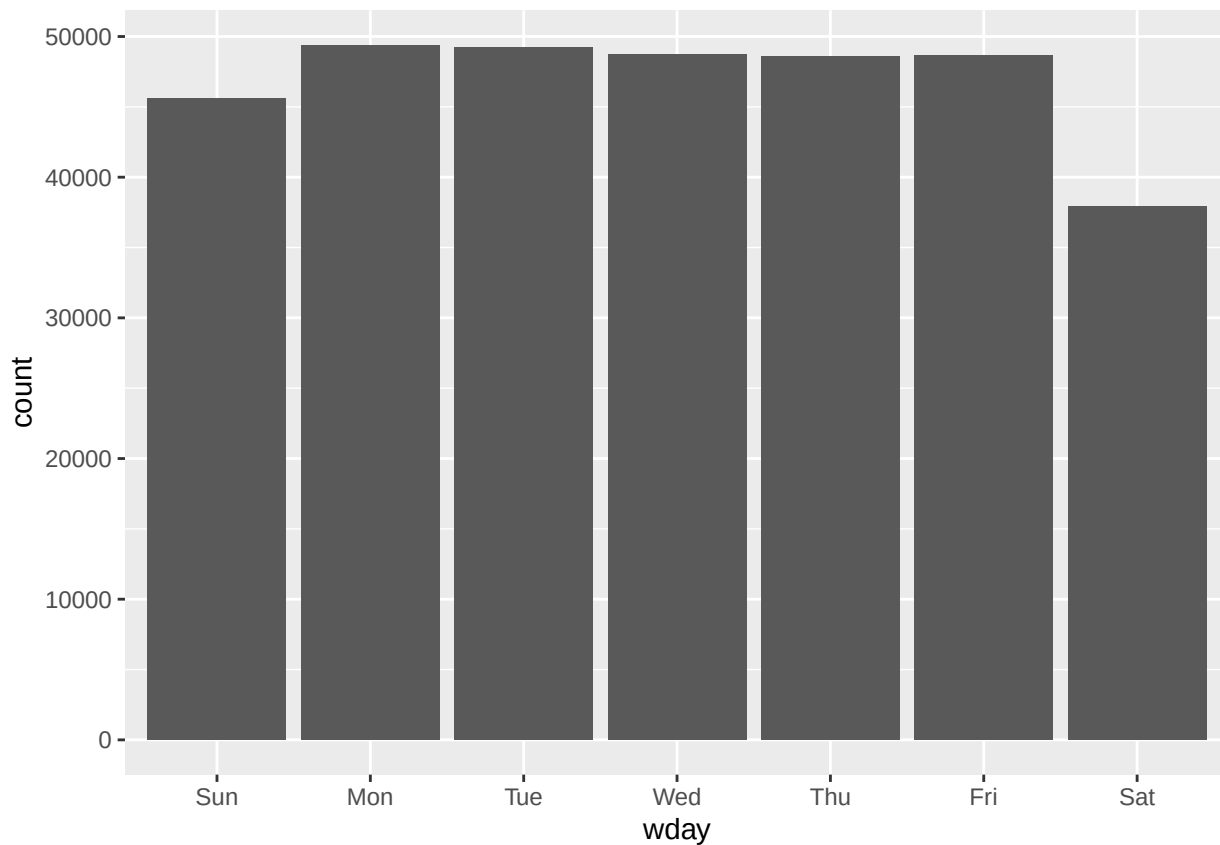
```
## [1] 189
```

```
wday(datetime)
```

```
## [1] 4
```

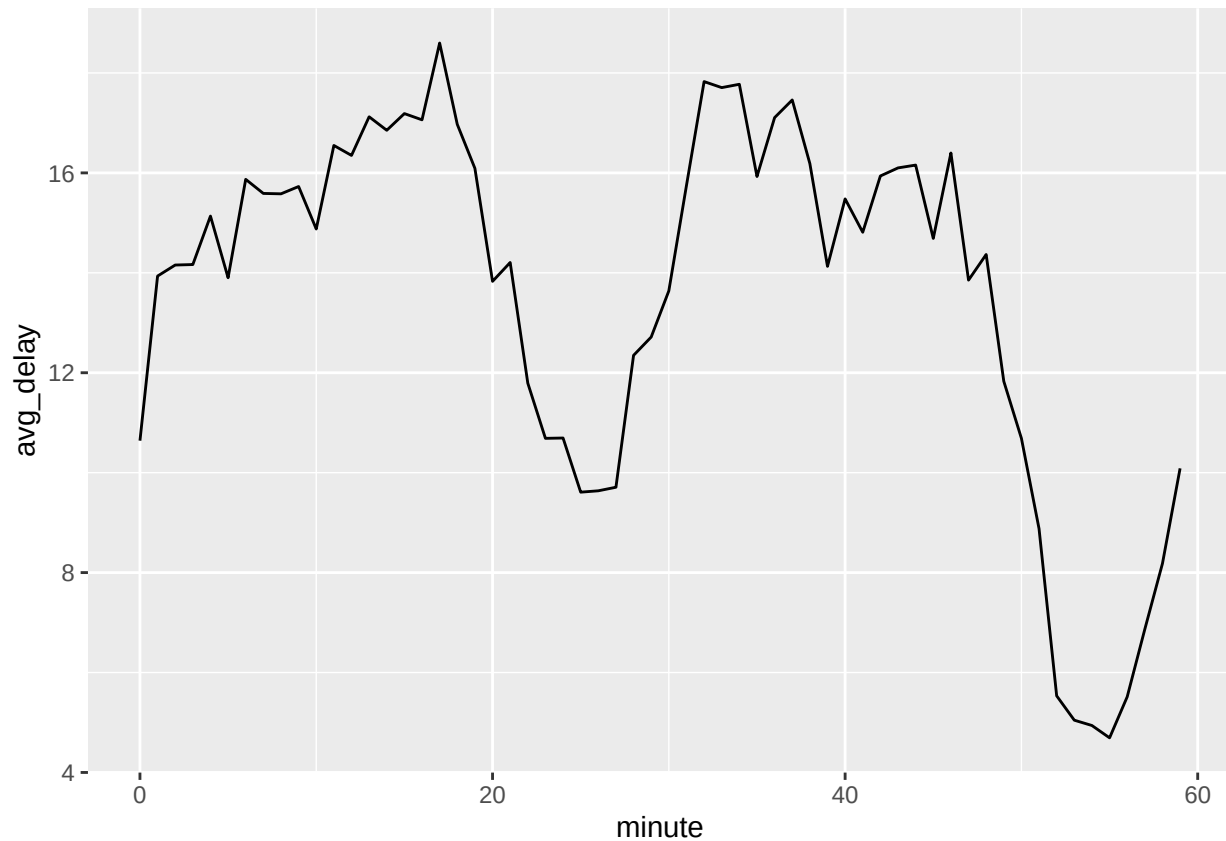
Example with dplyr:

```
flights_dt |>  
  mutate(wday = wday(dep_time, label = TRUE)) |>  
  ggplot(aes(x = wday)) +  
  geom_bar()
```



Example using `minute()` in `dplyr`:

```
flights_dt |>
  mutate(minute = minute(dep_time)) |>
  group_by(minute) |>
  summarize(
    avg_delay = mean(dep_delay, na.rm = TRUE),
    n = n()
  ) |>
  ggplot(aes(x = minute, y = avg_delay)) +
  geom_line()
```

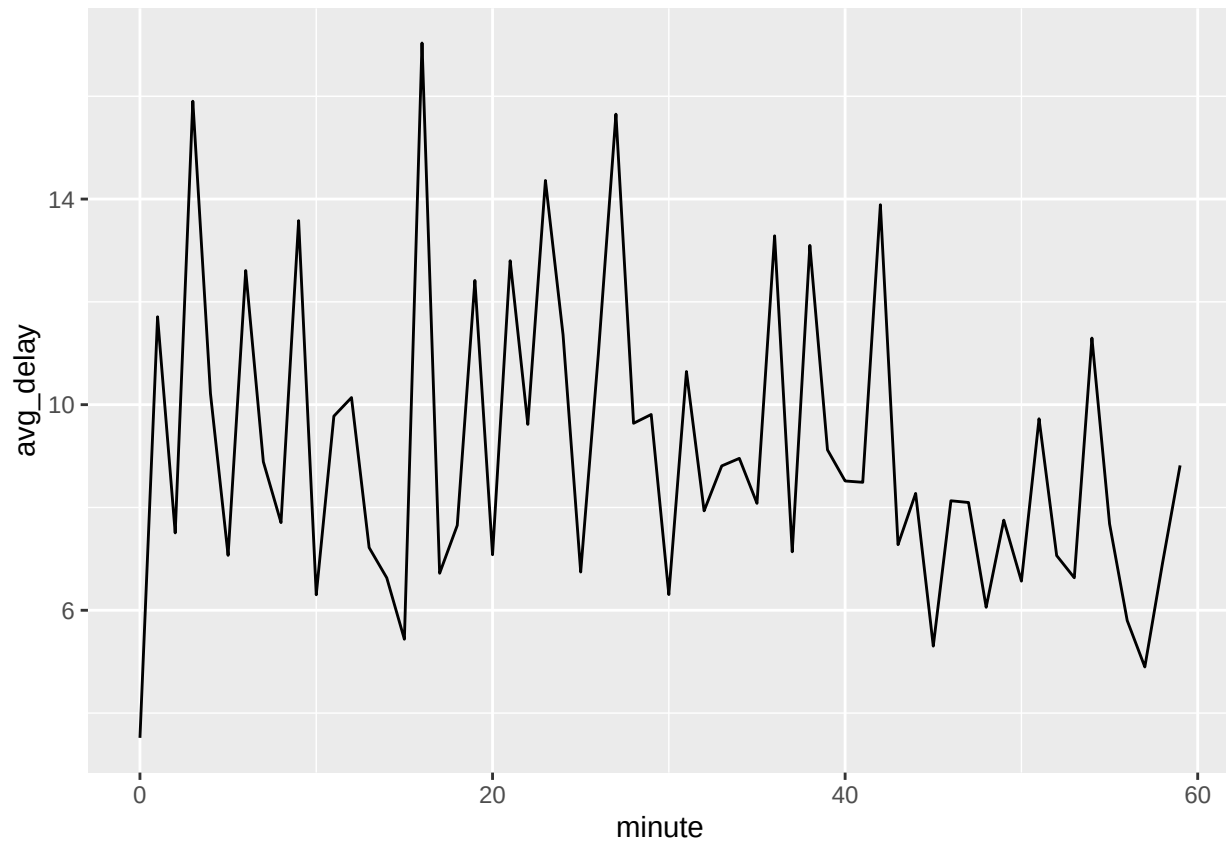


An interesting pattern can be observed here: flights leaving in 20-30 minutes or 50-60 minutes have much lower delays than the rest.

- Looking for patterns in `sched_dep_time`:

```
sched_dep <- flights_dt |>
  mutate(minute = minute(sched_dep_time)) |>
  group_by(minute) |>
  summarize(
    avg_delay = mean(arr_delay, na.rm = TRUE),
    n = n()
  )

ggplot(sched_dep, aes(x = minute, y = avg_delay)) +
  geom_line()
```

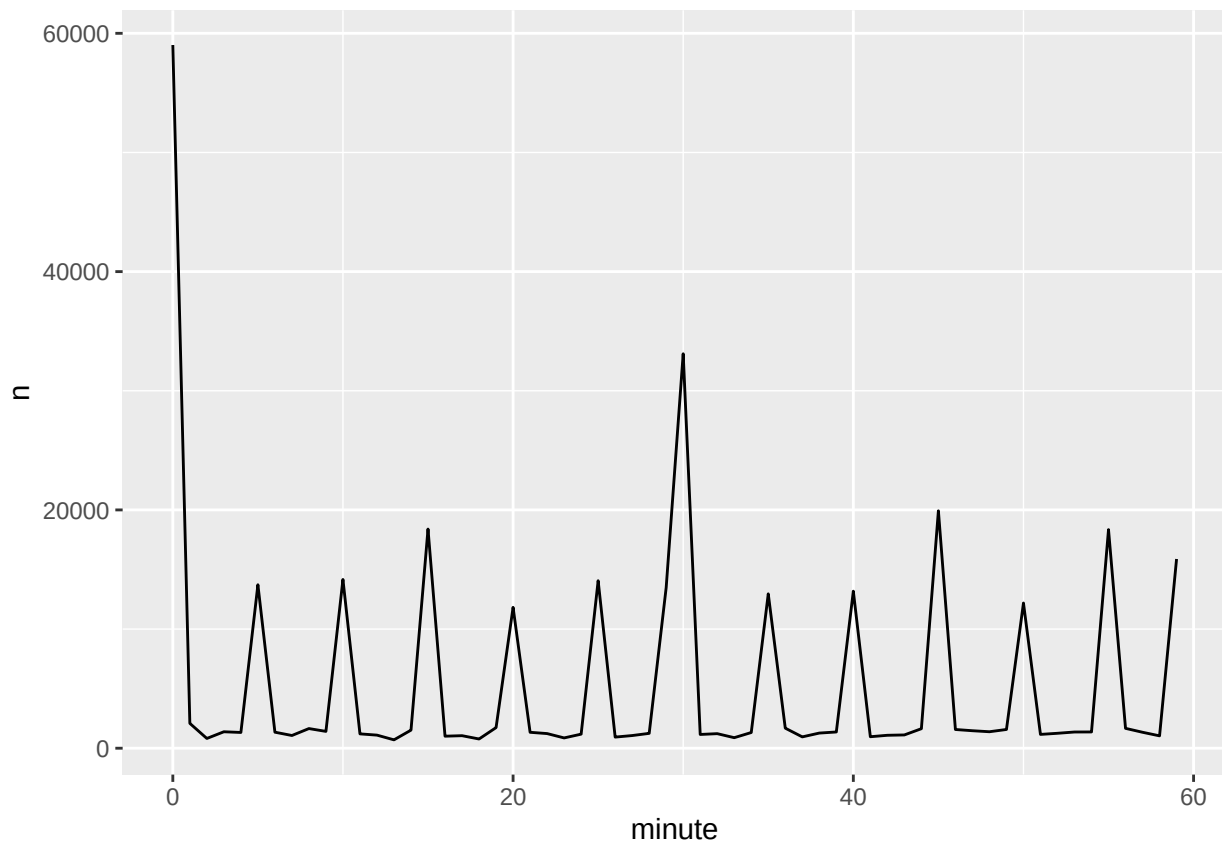


There is no pattern between `avg_delay` of actual departure times and minute.

The apparent pattern that we say earlier could be due to humans having bias towards flights leaving at “nice” departure times.

We should always be alert for this sort of pattern whenever we work with data that involves human judgement!

```
flights_dt |>
  mutate(sched_dep_time_minutes = minute(sched_dep_time)) |>
  group_by(sched_dep_time_minutes) |>
  summarize(
    minute = mean(sched_dep_time_minutes),
    n = n()
  ) |>
  ggplot(aes(x = minute, y = n)) +
  geom_line()
```

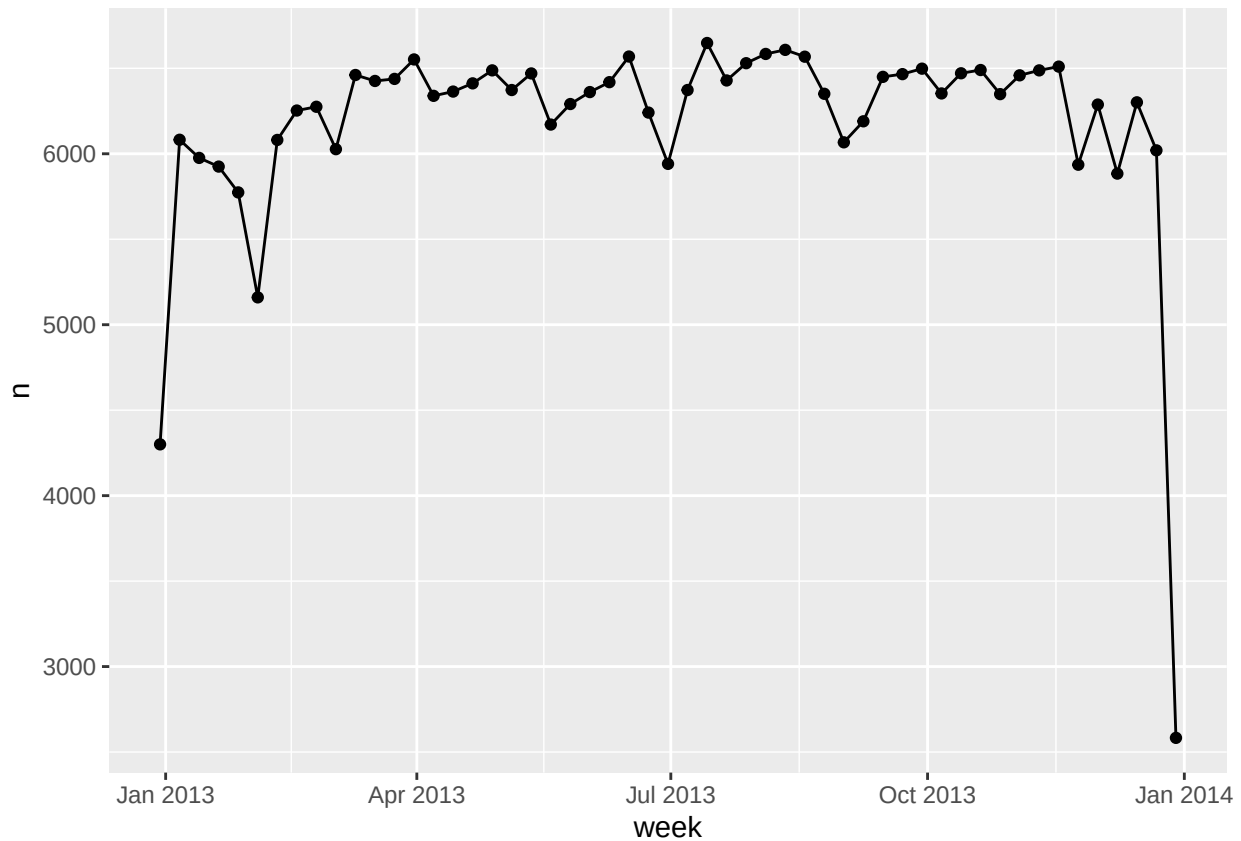


Looking at the plot of the number of flights scheduled to depart at each hour, we can see that there is a strong preference for round numbers and for numbers that are a multiple of five.

Rounding

- Options include `floor_date()`, `round_date()`, and `ceiling_date()`

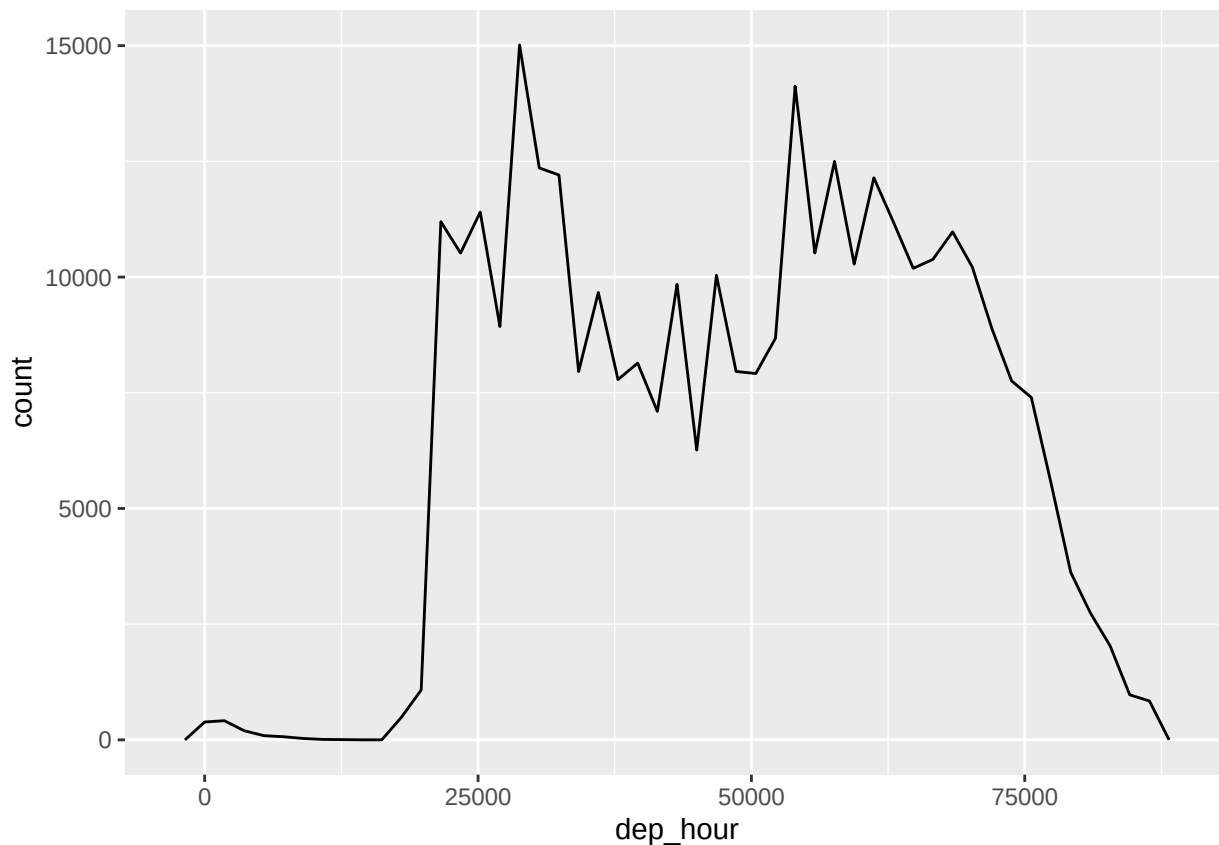
```
flights_dt |>
  count(week = floor_date(dep_time, "week")) |>
  ggplot(aes(x = week, y = n)) +
  geom_point() +
  geom_line()
```



Example showing the distribution of flights across the course of a day:

```
flights_dt |>
  mutate(dep_hour = dep_time - floor_date(dep_time, "day")) |>
  ggplot(aes(x = dep_hour)) +
  geom_freqpoly(binwidth = 60 * 30)
```

```
## Don't know how to automatically pick scale for object of type <difftime>.
## Defaulting to continuous.
```



Modifying components

- This method is useful when cleaning data that has clearly incorrect dates.

```
datetime <- ymd_hms("2026-07-08 12:34:56")
```

```
year(datetime) <- 2030
```

```
month(datetime) <- 01
```

```
hour(datetime) <- hour(datetime) + 1
```

```
dt
```

```
## function (x, df, ncp, log = FALSE)
## {
##   if (missing(ncp))
##     .Call(C_dt, x, df, log)
##   else .Call(C_dnt, x, df, ncp, log)
## }
## <bytecode: 0x556e6c45afa8>
## <environment: namespace:stats>
```

- Rather than modifying date-time one component at a time, we can use `update()` to modify multiple components in one go:

```
update(datetime, year = 2030, month = 2, mday = 2, hour = 2)
```

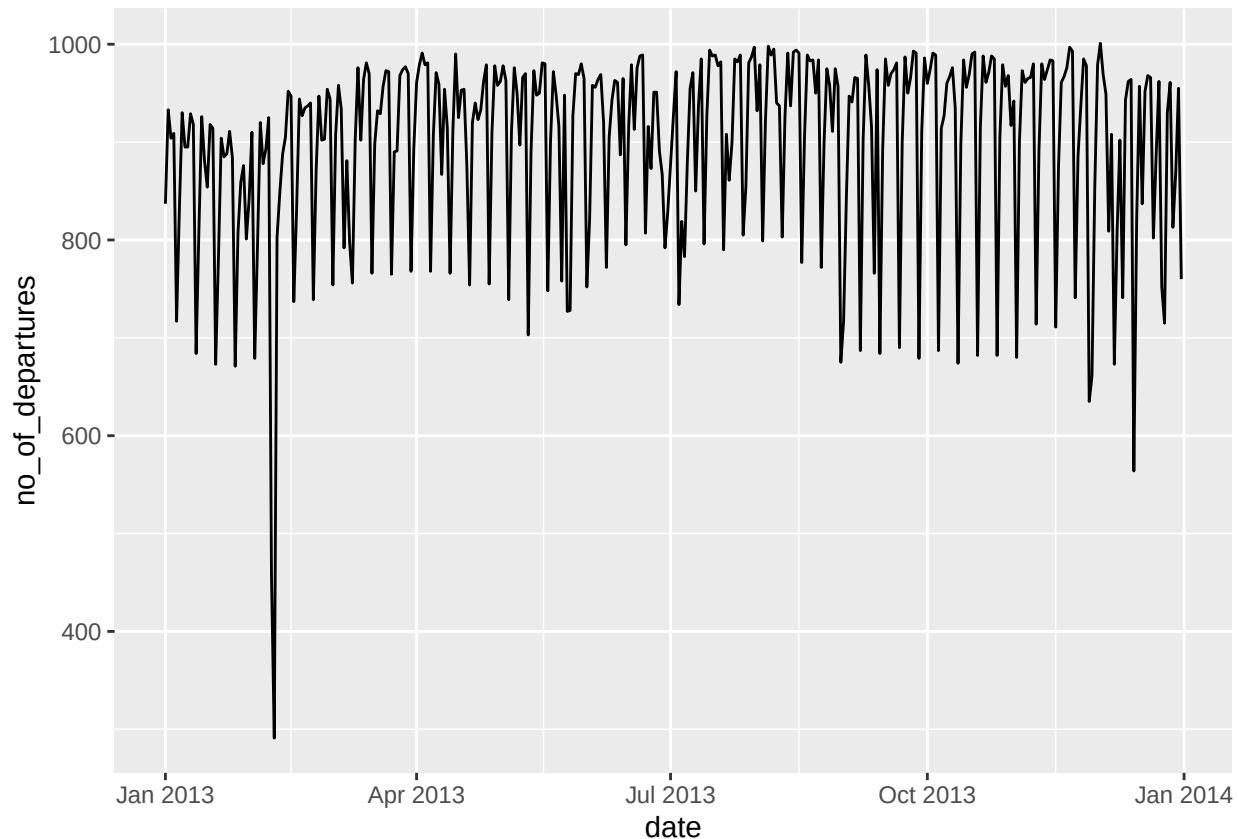
```
## [1] "2030-02-02 02:34:56 UTC"
```

Exercises

1. How does the distribution of flight times within a day change over the course of the year?

Ans.

```
flights_dt |>
  mutate(date = date(dep_time)) |>
  group_by(date) |>
  summarize(no_of_departures = n()) |>
  ggplot(aes(x = date, y = no_of_departures)) +
  geom_line()
```



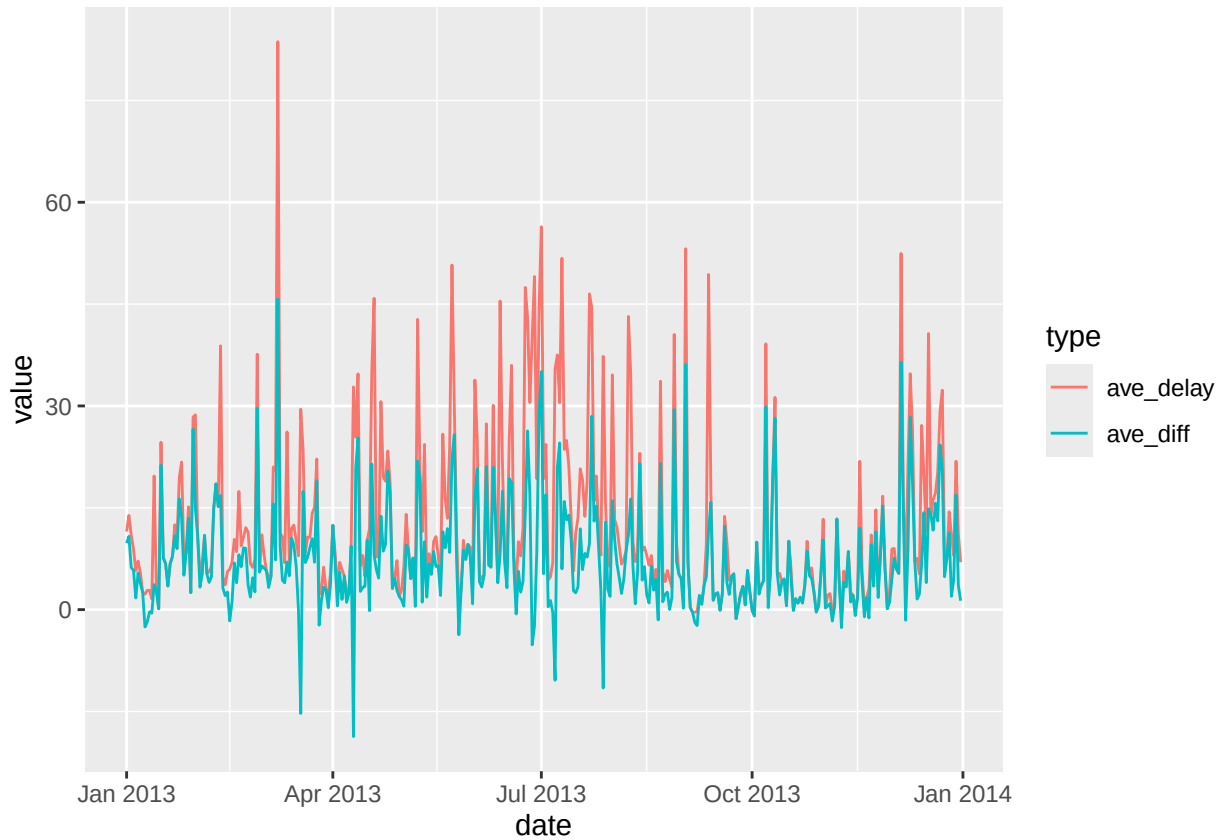
There seems to be a weekly and seasonal pattern.

2. Compare `dep_time`, `sched_dep_time`, and `dep_delay`. Are they consistent? Explain your findings.

Ans.

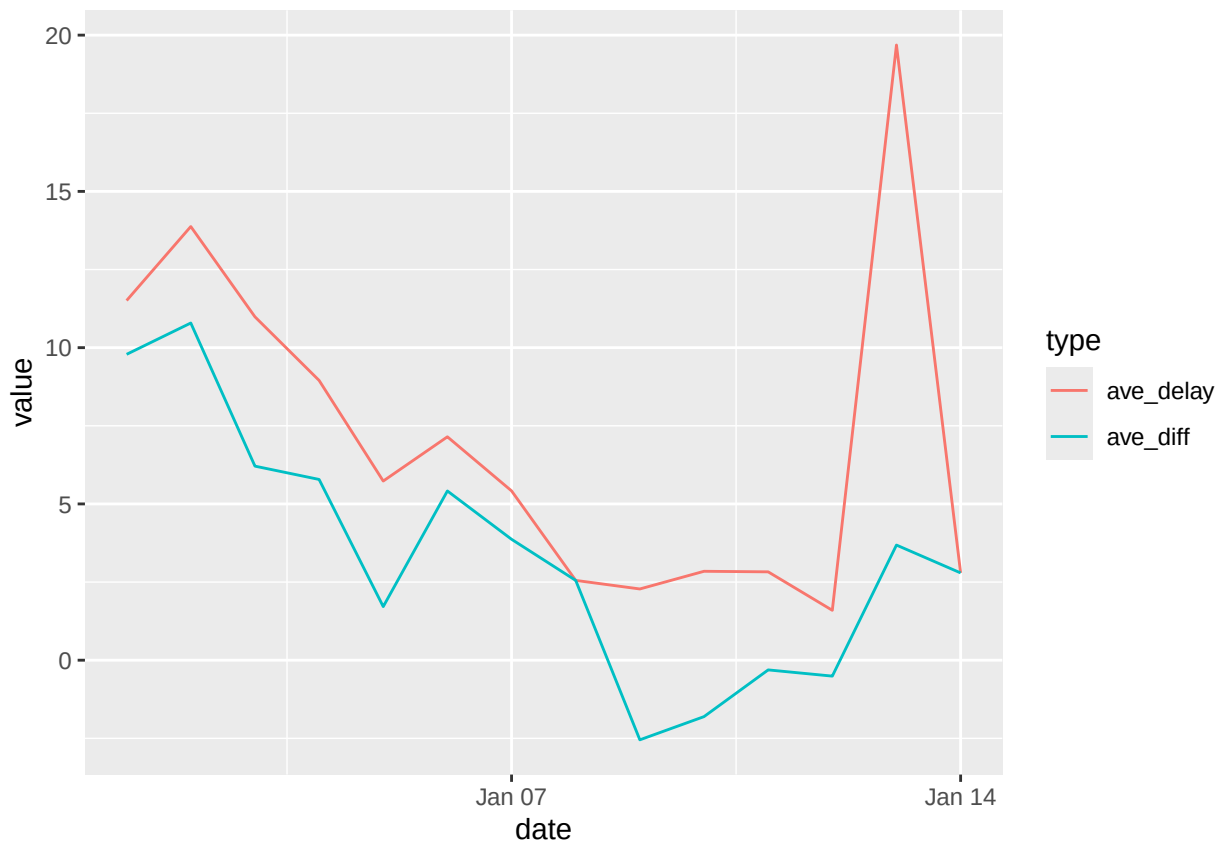
```
data2 <- flights_dt |>
  mutate(
    date = date(dep_time),
    diff = dep_time - sched_dep_time
  ) |>
  group_by(date) |>
  summarize(
    ave_diff = as.numeric(mean(diff)) / 60,
    ave_delay = mean(dep_delay)
  ) |>
  pivot_longer(cols = c("ave_diff", "ave_delay"), names_to = "type")
```

```
data2 |> ggplot(aes(x = date, y = value, color = type)) +  
  geom_line()
```



The delay and difference between departure time and scheduled departure time is pretty consistent (the delay and diff move together), but in many cases, their discrepancy is big. zooming in,

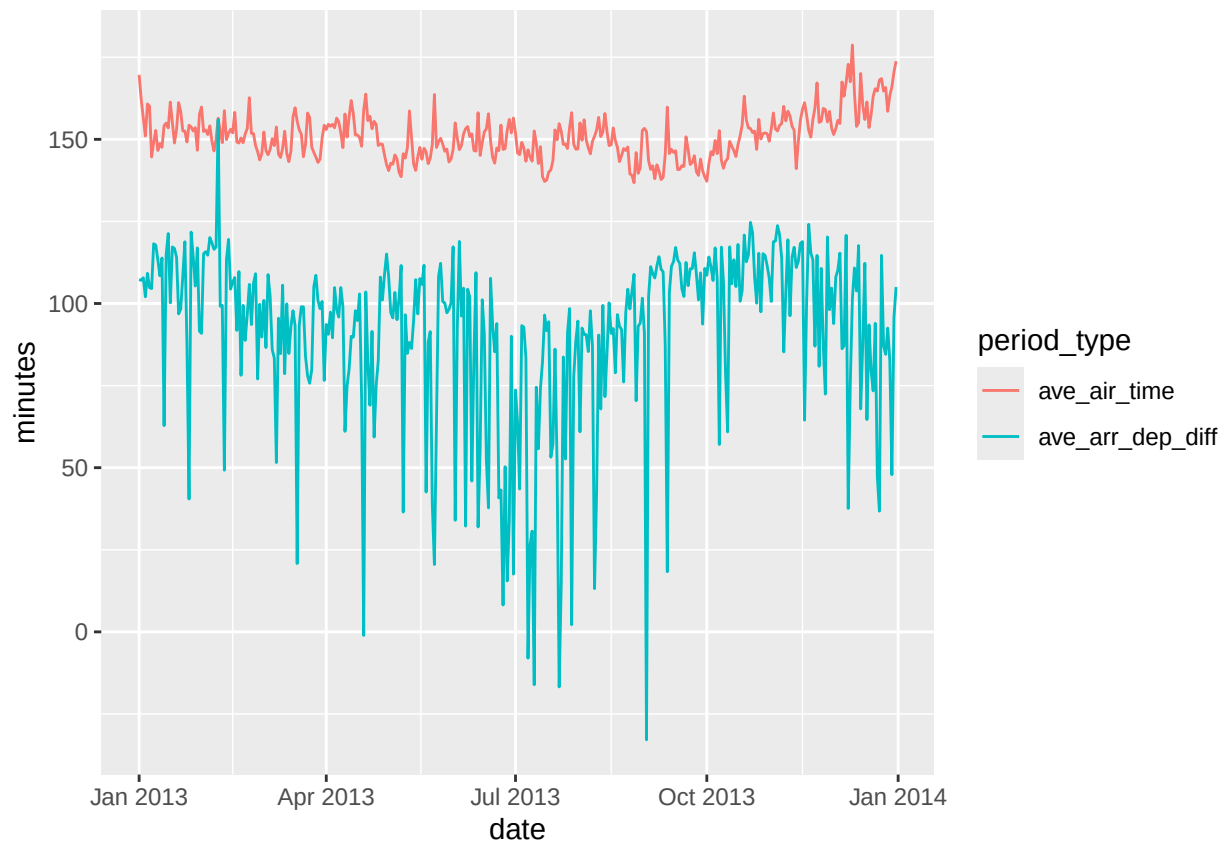
```
data2 |>  
  filter(date < as.Date("2013-01-15")) |>  
  ggplot(aes(x = date, y = value, color = type)) +  
  geom_line()
```



ave_delay and ave_diff move together consistently, but the gap between them are not very consistent.

3. Compare air_time with the duration between the departure and arrival. Explain your findings. (Hint: consider the location of the airport.)

```
flights_dt |>
  filter(!is.na(air_time)) |>
  mutate(
    date = date(dep_time),
    arr_dep_diff = as.numeric(arr_time - dep_time)
  ) |>
  group_by(date) |>
  reframe(
    ave_arr_dep_diff = mean(arr_dep_diff),
    ave_air_time = ave(air_time)
  ) |>
  pivot_longer(
    cols = c("ave_arr_dep_diff", "ave_air_time"),
    names_to = "period_type"
  ) |>
  ggplot(aes(x = date, y = value, color = period_type)) +
  geom_line() +
  labs(
    y = "minutes"
  )
```



Another attempt:

```
ave_at_diff_ratios <- flights_dt |>
  filter(!is.na(air_time)) |>
  select(dest, dep_time, arr_time, air_time) |>
  mutate(
    diff = as.numeric(arr_time - dep_time),
    date = as.Date(dep_time),
    at_diff_ratio = air_time / diff
  ) |>
  group_by(dest) |>
  summarize(ave_at_diff_ratio = mean(at_diff_ratio)) |>
  arrange(desc(ave_at_diff_ratio))
```

```
head(ave_at_diff_ratios)
```

```
## # A tibble: 6 x 2
##   dest ave_at_diff_ratio
##   <chr>         <dbl>
## 1 ANC             2.05
## 2 HNL             2.05
## 3 PSP             1.93
## 4 LAS             1.91
## 5 PHX             1.90
## 6 SNA             1.87
```

```
tail(ave_at_diff_ratios)
```

```
## # A tibble: 6 x 2
```

```
##   dest  ave_at_diff_ratio
##   <chr>          <dbl>
## 1 PVD            0.572
## 2 BWI            0.560
## 3 BDL            0.528
## 4 PHL            0.497
## 5 BQN            0.294
## 6 PSE            0.213
```

The discrepancy between `air_time` and the `diff` (the actual time difference between arrival times and departure times) is also affected by which time zone the destination belongs to. The more timezone the plane has to cross westward, the later the arrival time becomes.

4. How does the average delay time change over the course of a day? Should you use `dep_time` or `sched_dep_time`? Why?
5. On what day of the week should you leave if you want to minimize the chance of delay?

```
flights_dt |>
  mutate(
    date = date(dep_time),
    day = wday(date, label = TRUE)
  ) |>
  group_by(day) |>
  summarize(ave_dep_delay = mean(dep_delay)) |>
  arrange(ave_dep_delay)
```

```
## # A tibble: 7 x 2
##   day    ave_dep_delay
##   <ord>          <dbl>
## 1 Sat            7.61
## 2 Tue           10.6
## 3 Sun           11.5
## 4 Wed           11.7
## 5 Fri           14.7
## 6 Mon           14.7
## 7 Thu           16.1
```

You should leave on a Saturday to minimize the chance of delay.

6. What makes the distribution of `diamonds$carat` and `flight$sched_dep_time` similar?

Ans. Both data exhibit a form of discretization arising from human's preference for "nice" values like half-hour intervals, multiples of five, or certain carat values, heights in feet-inches, etc.

7. Confirm our hypothesis that the early departures of flights in minutes 20-30 and 50-60 are caused by scheduled flights that leave early. Hint: create a binary variable that tells you whether or not a flight was delayed.

Ans.

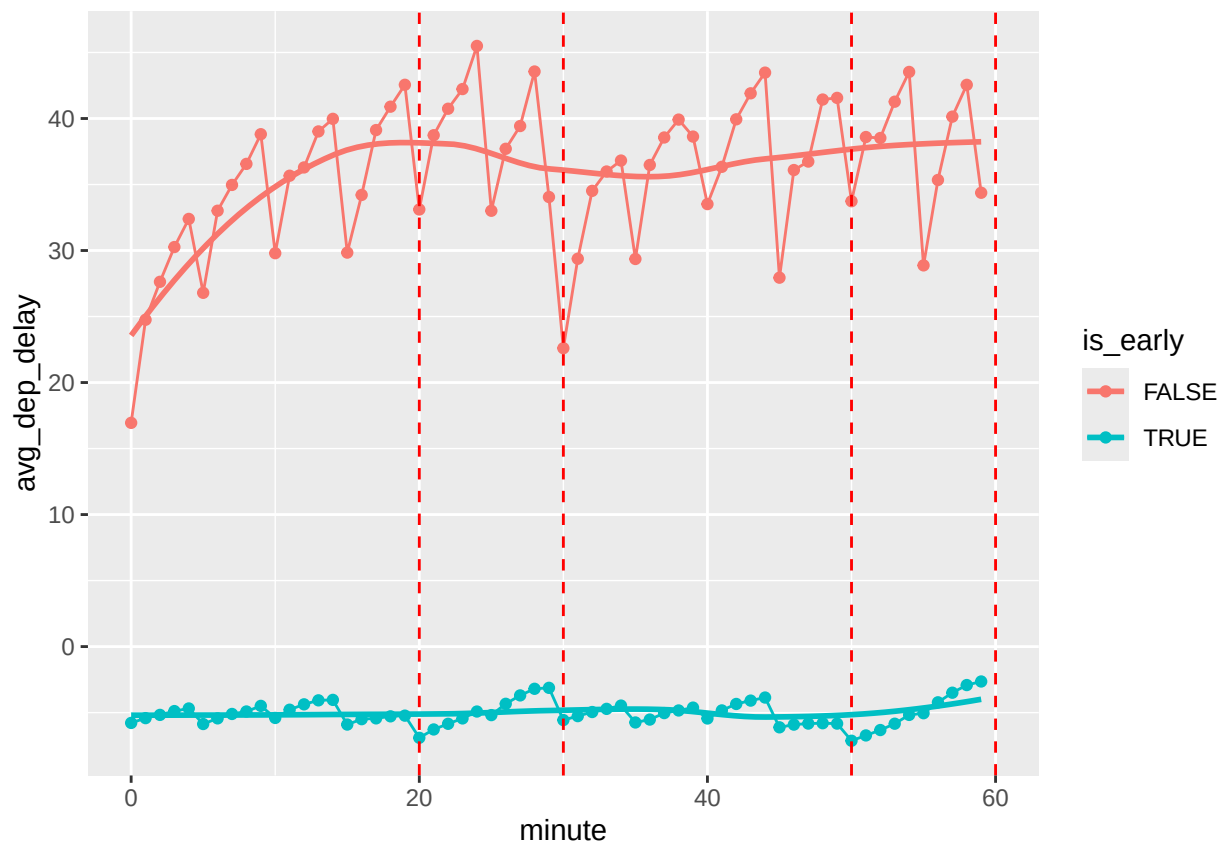
```
flights_dt |>
  mutate(
    minute = minute(dep_time),
    is_early = dep_delay < 0
  ) |>
  group_by(minute, is_early) |>
  summarize(
    avg_dep_delay = mean(dep_delay, na.rm = TRUE),
```

```

n = n()
) |>
ggplot(aes(x = minute, y = avg_dep_delay, color = is_early)) +
  geom_point() +
  geom_line() +
  geom_smooth(se = FALSE) +
  geom_vline(xintercept = 20, color = "red", lty = 2) +
  geom_vline(xintercept = 30, color = "red", lty = 2) +
  geom_vline(xintercept = 50, color = "red", lty = 2) +
  geom_vline(xintercept = 60, color = "red", lty = 2)

```

`summarise()` has grouped output by 'minute'. You can override using the
 ## `.groups` argument.
 ## `geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



Time spans

Classes that represent time spans: - **Durations:** represent an exact number of seconds.
 - **Periods:** represent human units like weeks and months. - **Intervals:** represent a starting and ending point.

Durations

- durations always record the time span in seconds.

```

# How old is Hadley?
h_age = today() - ymd("1979-10-14")
h_age

```

```
## Time difference of 16730 days
```

```
as.duration(h_age)
```

```
## [1] "1445472000s (~45.8 years)"
```

Convenient constructors for durations:

```
dseconds(15)
```

```
## [1] "15s"
```

```
minutes(10)
```

```
## [1] "10M 0S"
```

```
dhours(c(12, 14))
```

```
## [1] "43200s (~12 hours)" "50400s (~14 hours)"
```

```
ddays(0:5)
```

```
## [1] "0s" "86400s (~1 days)" "172800s (~2 days)"
```

```
## [4] "259200s (~3 days)" "345600s (~4 days)" "432000s (~5 days)"
```

```
dweeks(3)
```

```
## [1] "1814400s (~3 weeks)"
```

```
dyears(1)
```

```
## [1] "31557600s (~1 years)"
```

- add and multiply durations:

```
2 * dyears(1)
```

```
## [1] "63115200s (~2 years)"
```

```
dyears(1) + dweeks(12) + dhours(15)
```

```
## [1] "38869200s (~1.23 years)"
```

- add and subtract durations to and from days:

```
tomorrow = today() + ddays(1)
```

```
last_year = today() - dyears(1)
```

- Durations represent an exact number of seconds:

```
one_am <- ymd_hms("2026-03-08 01:00:00", tz = "America/New_York")
```

```
one_am
```

```
## [1] "2026-03-08 01:00:00 EST"
```

```
one_am + ddays(1)
```

```
## [1] "2026-03-09 02:00:00 EDT"
```

Why? Because March 8 has only 23 hours because of DST, so if we add a full days worth of seconds, we end up with a different time.

Periods

- more intuitive since they don't have fixed lengths:

```

one_am
## [1] "2026-03-08 01:00:00 EST"
one_am + days(1)
## [1] "2026-03-09 01:00:00 EDT"
hours(c(12, 24))
## [1] "12H 0M 0S" "24H 0M 0S"
days(7)
## [1] "7d 0H 0M 0S"
months(1:6)
## [1] "1m 0d 0H 0M 0S" "2m 0d 0H 0M 0S" "3m 0d 0H 0M 0S" "4m 0d 0H 0M 0S"
## [5] "5m 0d 0H 0M 0S" "6m 0d 0H 0M 0S"

```

- adding and multiplying:

```

10 * (months(6) + days(1))
## [1] "60m 10d 0H 0M 0S"
days(50) + hours(25) + minutes(2)
## [1] "50d 25H 2M 0S"

```

- Compared to durations, periods are more likely to do what you expect:

```

# a leap year
ymd("2024-01-01") + dyears(1)
## [1] "2024-12-31 06:00:00 UTC"
ymd("2024-01-01") + years(1)
## [1] "2025-01-01"

```

```

flights_dt |>
  filter(arr_time < dep_time) |>
  glimpse()

```

Fixing an oddity related to flights where some planes appear to arrive at their destination before they departed from New York City

```

## Rows: 10,633
## Columns: 9
## $ origin      <chr> "EWR", "JFK", "EWR", "EWR", "EWR", "LGA", "EWR", "JFK", ~
## $ dest        <chr> "BQN", "DFW", "TPA", "SJU", "SFO", "FLL", "MCO", "LAX", ~
## $ dep_delay   <dbl> 9, 59, -2, -6, 11, -10, 41, -7, 49, -9, 5, 2, 285, 24, ~
## $ arr_delay   <dbl> -4, NA, 9, -12, -14, -2, 43, -24, 28, -14, -14, 2, 246, ~
## $ dep_time    <dtm> 2013-01-01 19:29:00, 2013-01-01 19:39:00, 2013-01-01 2~
## $ sched_dep_time <dtm> 2013-01-01 19:20:00, 2013-01-01 18:40:00, 2013-01-01 2~
## $ arr_time    <dtm> 2013-01-01 00:03:00, 2013-01-01 00:29:00, 2013-01-01 0~
## $ sched_arr_time <dtm> 2013-01-01 00:07:00, 2013-01-01 21:51:00, 2013-01-01 2~
## $ air_time    <dbl> 192, NA, 159, 199, 354, 160, 143, 338, 152, 154, 189, 1~

```

We can see that these flights arrived on the following day, but the date information was not changed. We can fix this by adding `days(1)` to the arrival time of each overnight flight:

```
flights_dt <- flights_dt |>
  mutate(
    overnight = arr_time < dep_time,
    arr_time = arr_time + days(overnight),
    sched_arr_time = sched_arr_time + days(overnight)
  )
```

Checking:

```
flights_dt |>
  filter(arr_time < dep_time)
```

```
## # A tibble: 0 x 10
## #   i 10 variables: origin <chr>, dest <chr>, dep_delay <dbl>, arr_delay <dbl>,
## #     dep_time <dtm>, sched_dep_time <dtm>, arr_time <dtm>,
## #     sched_arr_time <dtm>, air_time <dbl>, overnight <lgl>
```

Now all the flights obey the laws of physics!

Intervals

When it comes to years, humans usually think of 365 days, but actually this is not exact:

```
dyears(1) / ddays(365)
```

```
## [1] 1.000685
```

```
years(1) / days(1)
```

```
## [1] 365.25
```

The answer should also change depending on whether the year is a leap year or not.

A better way for a more accurate measurement is to use an **interval**. Format for writing an interval is **start** `%--%` **end**:

```
y2023 <- ymd("2023-01-01") %--% ymd("2024-01-01")
y2024 <- ymd("2024-01-01") %--% ymd("2025-01-01")
```

```
y2023
```

```
## [1] 2023-01-01 UTC--2024-01-01 UTC
```

```
y2024
```

```
## [1] 2024-01-01 UTC--2025-01-01 UTC
```

Check by dividing these with `days()`:

```
y2023 / days(1)
```

```
## [1] 365
```

```
y2024 / days(1)
```

```
## [1] 366
```

Time zones

Determining the “current” time zone (actually, it is what R thinks your current time zone is) using `Sys.timezone()`:

```
Sys.timezone()
```

```
## [1] "Asia/Manila"
```

Showing the list of time zone names using `OlsonNames()`:

```
length(OlsonNames())
```

```
## [1] 599
```

```
head(OlsonNames())
```

```
## [1] "Africa/Abidjan"      "Africa/Accra"        "Africa/Addis_Ababa"
## [4] "Africa/Algiers"      "Africa/Asmara"       "Africa/Asmera"
```

Time zone maths:

```
# three time zone objects that represent the same instant in time:
x1 <- ymd_hms("2024-06-01 12:00:00", tz = "America/New_York")
x2 <- ymd_hms("2024-06-01 18:00:00", tz = "Europe/Copenhagen")
x3 <- ymd_hms("2024-06-02 04:00:00", tz = "Pacific/Auckland")

x1 - x2
```

```
## Time difference of 0 secs
```

```
x1 - x3
```

```
## Time difference of 0 secs
```

- Otherwise specified, `lubridate` always uses UTC (Coordinated Universal Time).
- When combining time zones with `c()`, time zones will be dropped and the time-dates will display in the time zone of the first element:

```
x4 <- c(x1, x2, x3)
x4
```

```
## [1] "2024-06-01 12:00:00 EDT" "2024-06-01 12:00:00 EDT"
## [3] "2024-06-01 12:00:00 EDT"
```

- Displaying combined time-dates in another time zone:

```
x4a <- with_tz(x4, tzzone = "Australia/Lord_Howe")
x4a
```

```
## [1] "2024-06-02 02:30:00 +1030" "2024-06-02 02:30:00 +1030"
## [3] "2024-06-02 02:30:00 +1030"
```

```
x4a - x4
```

```
## Time differences in secs
```

```
## [1] 0 0 0
```

Fixing incorrect time zone using `force_tz()`:

```
x4b <- force_tz(x4, tzzone = "Australia/Lord_Howe")
x4b
```

```
## [1] "2024-06-01 12:00:00 +1030" "2024-06-01 12:00:00 +1030"
```

```
## [3] "2024-06-01 12:00:00 +1030"
```

```
x4b - x4
```

```
## Time differences in hours
```

```
## [1] -14.5 -14.5 -14.5
```